

GRANOSCAR MASTER STACK

RECURSOS GRATUITOS + OPTIMIZACIÓN DE TOKENS

Repositorios GitHub verificados de diseño web, gráfico, imagen,
seguridad, software Windows y apps móviles
+ sistema de contexto persistente para Claude Code y Codex CLI

Preparado para:

Jean Carlos — YANKYFILMS / MEDIAPLAYPROMO.COM
GRANOSCAR · i7-13700K · RTX 4070 Ti 12GB · 96GB RAM

ÍNDICE

1. Cómo usar este documento (lectura obligatoria para Codex/Claude)
2. Sistema de optimización de tokens — Claude Code
3. Sistema de optimización de tokens — OpenAI Codex
4. Protocolo de herencia de contexto entre proyectos / agentes
5. Repositorios — Diseño Web & Frontend
6. Repositorios — Diseño Gráfico, UI/UX, Iconos e Imágenes
7. Repositorios — Seguridad Web (defensiva, para SaaS propios)
8. Repositorios — Programación de Software para Windows
9. Repositorios — Apps Móviles (Android/iOS)
10. Software libre de Edición (imagen, foto, vídeo, vectores)
11. Plantilla maestra AGENTS.md / CLAUDE.md para copiar y pegar
12. Checklist final de instalación en GRANOSCAR

01 CÓMO USAR ESTE DOCUMENTO

Instrucciones para ti y para cualquier agente de IA (Codex, Claude Code) que lea este PDF

Este documento está diseñado para que se lo pegues directamente a un Codex nuevo, a una nueva sesión de Claude Code, o a cualquier agente de IA al inicio de un proyecto. Contiene dos cosas distintas que no debes mezclar:

A) Una librería curada de repositorios GitHub gratuitos — verificados, activos y de calidad real (no listas genéricas con enlaces muertos) para diseño web, diseño gráfico, imágenes, seguridad, software Windows y apps móviles.

B) Un sistema de configuración para que Codex y Claude Code gasten el mínimo de tokens posible, no releen los mismos archivos repetidamente, y para que un agente nuevo pueda entender un proyecto tuyo anterior sin que tengas que re-explicarlo todo desde cero.

Para la parte B necesitas crear, en la raíz de cada uno de tus proyectos (NEXFRAME FILMS, AFFILIX, YF Studio Pro, PUBLIMASTER IA, etc.), un archivo llamado **AGENTS.md** (lo usa Codex) y opcionalmente un **CLAUDE.md** (lo usa Claude Code). Ambos pueden contener exactamente lo mismo. En la Sección 11 tienes la plantilla lista para copiar y rellenar.

REGLA DE ORO: un agente de IA no "recuerda" nada entre sesiones a menos que tú lo escribas en un archivo dentro del repositorio. La memoria no vive en la cuenta, vive en el código. Por eso todo este sistema se basa en archivos .md dentro de cada proyecto, no en la nube ni en configuración de cuenta.

02 OPTIMIZACIÓN DE TOKENS — CLAUDE CODE

Cómo evitar que Claude Code relea los mismos archivos y agote el contexto

Claude Code reconstruye contexto en cada sesión a partir de varias fuentes: el archivo `CLAUDE.md`, el historial de la conversación, los archivos que lee, y la salida de las herramientas que ejecuta. El problema de "gastar muchos tokens" casi siempre es contexto mal gestionado, no un problema de la IA en sí.

2.1 — El archivo CLAUDE.md (la pieza más importante)

El `CLAUDE.md` se carga ANTES de cada turno y permanece en el contexto durante toda la sesión, se use o no. Un `CLAUDE.md` de 5.000 tokens te cuesta 5.000 tokens en cada mensaje, tengas 2 mensajes o 200 en la conversación. Por eso la regla de Anthropic es: máximo 200 líneas, solo hechos que Claude no puede deducir leyendo el código.

Un caso real medido: un `CLAUDE.md` de 3.847 tokens (el típico que se genera dejando que Claude documente todo lo que se encuentra durante el onboarding) se redujo a 312 tokens quitando todo lo que no era estrictamente necesario. Resultado: 91,9% menos contexto gastado, sin pérdida de calidad en las respuestas.

Qué SÍ va en el CLAUDE.md:

- Comandos de build, test y arranque del proyecto (npm run dev, etc.)
- Gestor de paquetes a usar (npm/pnpm/yarn) y por qué
- Reglas de arquitectura que el código no deja ver a simple vista (ej. "todo acceso a BD pasa por src/db/repository.py")
- Carpetas que Claude NUNCA debe tocar (vendor/, node_modules/, build/)
- Estilo de commits y convenciones de nombres

Qué NO va en el CLAUDE.md:

- Historial de decisiones de diseño o reuniones (eso va en docs/decisions/, no en el contexto siempre-cargado)
- Guías largas de implementación (usa @includes a archivos aparte, ver 2.3)
- Explicaciones de cómo funciona el código (Claude ya lo lee solo)

2.2 — Skills, Subagents y Hooks: la diferencia clave

Claude Code tiene tres mecanismos de extensión que mucha gente confunde. Usarlos bien es la diferencia entre una sesión que se queda sin contexto a la hora y una que dura todo el día:

Mecanismo	Qué hace	Cuándo usarlo
Skills (.claude/skills/)	Procedimientos reutilizables que se cargan SOLO cuando hacen falta (no siempre)	Tareas repetitivas: generar PDF, revisar código, formato de commits
Subagents (.claude/agents/)	Trabajadores con su PROPIA ventana de contexto, separada de la principal	Búsquedas largas, auditorías, exploración de código que no quieres que ensucie tu conversación principal

Hooks (settings.json)	Scripts deterministas que se disparan en eventos (antes de escribir un archivo, antes de ejecutar un comando)	Reglas que deben cumplirse el 100% de las veces (seguridad, bloquear 'rm -rf', forzar formato)
--------------------------	---	--

Regla práctica: si Claude va a investigar algo que vas a necesitar UNA vez (leer documentación externa, explorar 50 archivos buscando una función), usa un subagent — su exploración detallada no contamina tu conversación principal, solo te devuelve el resultado final.

2.3 — Evitar que relea los mismos archivos 50 veces

Esto es justo lo que pediste. Tres mecanismos lo resuelven:

a) @includes con alcance limitado

En vez de pegar toda la documentación de arquitectura en CLAUDE.md, usa referencias (@includes) a archivos específicos en docs/. Claude solo carga ese archivo cuando realmente está trabajando en esa parte del proyecto, no en cada turno.

b) .claudeignore

En proyectos grandes (como tu NEXUS o YF Studio Pro con múltiples paneles), crea un archivo .claudeignore en la raíz y en cada subcarpeta grande. Esto evita que Claude indexe carpetas enteras que no son relevantes para la tarea actual (assets, builds, node_modules, .next, dist).

c) /compact y un protocolo de compactación explícito

Cuando una sesión se alarga, usa /compact para resumir el contexto anterior en vez de cargarlo completo. Pero el resumen automático puede perder detalles importantes. La solución es decirle explícitamente a Claude QUÉ preservar al compactar, dentro del propio CLAUDE.md:

```
## Compact Instructions
Cuando resumas esta conversación:
- Conserva todos los cambios de API y su razón
- Conserva mensajes de error y sus soluciones
- Mantén la lista de archivos modificados
- Resume brevemente los intentos de exploración que no funcionaron
```

2.4 — Sesiones por fase, no sesiones maratón

El "context rot" (degradación de calidad) aparece típicamente después de 1-2 horas de sesión continua: Claude empieza a perder de vista instrucciones tempranas y comete errores que antes no cometía. Mejor: una sesión por unidad de trabajo coherente (una feature, un bug, un módulo), no una sesión que abarca todo un día de trabajo en NEXFRAME FILMS de una sola vez.

03 OPTIMIZACIÓN DE TOKENS — OPENAI CODEX

AGENTS.md, gestión de hilos, y verificación

Codex funciona de forma distinta a Claude Code en un punto clave: la mayoría de su contexto viene del PROPIO REPOSITORIO (archivos, AGENTS.md, skills), no de la conversación. Esto significa que optimizar tokens en Codex es, sobre todo, preparar bien tu repo y tu archivo de configuración — no escribir el prompt perfecto.

3.1 — AGENTS.md: la guía durable

AGENTS.md es el equivalente de Codex al CLAUDE.md. Se carga automáticamente en cada sesión dentro de ese repositorio — ya sea que lo ejecutes tú desde el CLI, desde la app de escritorio, o de forma automatizada desde un issue de GitHub. Una estructura recomendada para tus proyectos:

```
## Arquitectura
- Todo acceso a base de datos pasa por src/db/repository.py
- El módulo src/payments/ tiene su propio flujo de auth, separado de src/auth/
- Ninguna dependencia nueva en producción sin antes proponerla

## Antes de marcar una tarea como completa
- Ejecutar: npm run test && npm run lint && npm run typecheck
- No aceptar el resultado sin correr estos tres comandos
```

3.2 — Gestión de hilos (threads)

Las sesiones de Codex no son solo historial de chat: son hilos de trabajo que acumulan contexto, decisiones y acciones. Recomendaciones oficiales de OpenAI:

- Un hilo por unidad de trabajo coherente. Si sigues en el mismo problema, queda en el mismo hilo (preserva el rastro de razonamiento).
- Usa /compact cuando el hilo se alarga, para obtener una versión resumida del contexto anterior.
- Usa /agent para cambiar entre agentes activos si corres varios en paralelo.

3.3 — REQUIREMENTS.md, AGENT_TASKS.md y Backlog.md

Para flujos multi-sesión o multi-rol (exactamente tu caso: vas y vienes entre NEXFRAME, AFFILIX, YF Studio Pro), la práctica recomendada es mantener tres archivos en la raíz de cada proyecto:

- **REQUIREMENTS.md** — la especificación completa del sistema (tu PDF maestro Codex-ready ya cumple este rol)
- **AGENT_TASKS.md** — qué tarea está haciendo cada sesión/agente y qué archivos debe producir, sin que tengas que re-explicarlo
- **Backlog.md** — tareas pendientes del proyecto. Mantenlo corto; archiva lo completado por separado para que el backlog activo siga siendo ligero

Estos tres archivos funcionan como "entregas" estructuradas entre sesiones: cada agente sabe exactamente qué hacer y qué debe producir sin necesidad de que le re-expliques el contexto previo.

3.4 — Modo sandbox y permisos (evita fricción = evita reintentos = ahorra tokens)

Si Codex te pide aprobación constantemente para operaciones rutinarias en un repo de confianza, relaja la política de aprobación en config.toml. Cada vez que Codex se bloquea y reintenta una operación, eso es contexto y tokens gastados sin avanzar trabajo real.

04 HERENCIA DE CONTEXTO ENTRE PROYECTOS

Cómo lograr que un Codex/Claude nuevo entienda un proyecto anterior tuyo sin re-explicar nada

Esto es lo que pediste exactamente: que cuando abras un proyecto nuevo (o vuelvas a uno antiguo) con OTRA sesión de Codex o Claude, el agente lea todo y actúe "como si él mismo lo hubiera creado", sin que tengas que volver a contar la historia completa cada vez.

4.1 — Por qué esto NO funciona solo con la cuenta

Es importante que entiendas esto para no perder tiempo: la memoria de Codex/Claude entre sesiones distintas NO vive en tu cuenta de OpenAI/Anthropic de forma automática salvo en apps con "memoria de proyecto" persistente (como la app de escritorio de Codex). La forma fiable y que SIEMPRE funciona, sea cual sea la sesión, dispositivo o incluso modelo, es: la memoria vive DENTRO DEL REPOSITORIO, como archivos versionados en Git.

4.2 — El protocolo de 4 archivos (úsalo en TODOS tus proyectos)

AGENTS.md / CLAUDE.md

Reglas durables del proyecto: arquitectura, comandos, convenciones, qué no tocar. Se carga SIEMPRE automáticamente.

PROJECT_STATE.md

Snapshot del estado actual: qué está hecho, qué está a medias, qué decisiones de diseño se tomaron y por qué. Se actualiza al final de cada sesión importante.

AGENT_TASKS.md

La tarea concreta de la sesión actual: qué hay que construir, qué archivos tocar, qué archivos producir.

Backlog.md

Lista corta de pendientes activos. Se archiva lo completado aparte para que esto no crezca sin límite.

4.3 — El prompt de "arranque heredado" (cópialo al abrir cualquier sesión nueva)

Pega esto al inicio de cualquier sesión nueva de Codex o Claude Code, en la raíz del proyecto:

```
Antes de hacer nada, lee completos en este orden: AGENTS.md, PROJECT_STATE.md, AGENT_TASKS.md y Backlog.md. Actúa como si tú mismo hubieras construido este sistema en sesiones anteriores: no preguntes por contexto que ya está documentado en esos archivos, no repitas exploración ya hecha, y continúa exactamente desde donde quedó AGENT_TASKS.md. Si algo no está claro después de leer los 4 archivos, pregunta solo eso, una vez.
```

Esto funciona porque obliga al agente a construir su propio entendimiento leyendo archivos reales del repo (barato en tokens, una sola vez al inicio) en vez de que tú narres oralmente toda la historia del proyecto (caro, impreciso y no quedará guardado para la próxima vez).

4.4 — Al CERRAR una sesión importante

Pide siempre, al final de una sesión donde se avanzó algo relevante:

```
Antes de terminar, actualiza PROJECT_STATE.md con: qué se completó esta sesión, qué decisiones de arquitectura se tomaron y por qué, y qué queda pendiente. Actualiza también Backlog.md quitando lo
```

completado y añadiendo lo nuevo que haya surgido.

Con este hábito, cualquier Codex nuevo que abras —incluso en otra cuenta, otra máquina, u otro modelo— puede "despertar" dentro del proyecto en segundos, leyendo 4 archivos cortos, en vez de que tú reescribas el historial completo cada vez que cambias de sesión.

05 REPOSITORIOS — DISEÑO WEB & FRONTEND

Verificados activos en 2026, con licencia libre para uso comercial salvo que se indique lo contrario

shadcn/ui [BASE FRONTEND]

github.com/shadcn-ui/ui

Componentes UI hermosos contruidos sobre Radix UI + Tailwind CSS. No es una librería instalable: copias el código fuente a tu proyecto y lo personalizas. Es el estándar de facto en 2026 para interfaces modernas en React/Next.js — ideal para los paneles de NEXUS, AFFILIX y YF Studio Pro.

Tailwind CSS

github.com/tailwindlabs/tailwindcss

Framework CSS utility-first. Es la base de casi todo el ecosistema moderno de diseño (shadcn, Radix, Headless UI lo asumen como dependencia). Permite mantener tu identidad visual dark cinematográfica (negro/rojo/ámbar) con tokens de diseño consistentes en todos tus proyectos.

Radix UI

github.com/radix-ui/primitives

Componentes primitivos sin estilo, accesibles, sobre los que se construye shadcn/ui. Maneja toda la lógica compleja (focus trap, teclado, ARIA) para que tú solo te preocupes del estilo visual.

Next.js [FRAMEWORK]

github.com/vercel/next.js

El framework React de producción más usado. Tus proyectos NEXFRAME FILMS, YF Studio Pro y AFFILIX ya lo usan; este repo es la fuente oficial para entender las novedades de cada versión 14/15.

Framer Motion / Motion

github.com/motiondivision/motion

La librería estándar de animaciones para React. Perfecta para transiciones cinematográficas tipo "brutal flow" que ya usas en YF Studio Pro.

Awesome Frontend Dev Bookmarks [REFERENCIA]

github.com/dypsilon/frontend-dev-bookmarks

Colección curada de recursos de desarrollo frontend: algoritmos, patrones de diseño, animación, diseño responsive y accesibilidad. Útil como referencia rápida cuando Codex necesita un patrón concreto sin reinventar la rueda.

Build Your Own X

github.com/codecrafters-io/build-your-own-x

Guías paso a paso para reconstruir tecnologías desde cero (tu propio Git, tu propio servidor HTTP, tu propio motor de bases de datos). Útil cuando quieres que Codex entienda CÓMO funciona algo por dentro antes de integrarlo en NEXFRAME o DocuForge AI.

Awesome (sindresorhus) [META-LISTA]

github.com/sindresorhus/awesome

El listado raíz de TODAS las listas "awesome-*" de GitHub. Si necesitas algo muy específico que no está en este PDF, este es el punto de partida para encontrarlo verificado.

Vite

github.com/vitejs/vite

Bundler/dev-server ultrarrápido. Si alguno de tus dashboards (NEXUS, MCP MAESTRO) usa frontend ligero sin necesidad de SSR de Next.js, Vite es la alternativa más rápida para desarrollo local.

HTML5 Boilerplate

github.com/h5bp/html5-boilerplate

Plantilla profesional para proyectos HTML estático, con buenas prácticas de performance, SEO y compatibilidad ya resueltas.
Útil para landing pages rápidas tipo YF Sessions.

06 DISEÑO GRÁFICO, UI/UX, ICONOS E IMÁGENES

Recursos gratuitos para producción visual sin depender de stock de pago

design-resources-for-developers [IMPRESINDIBLE]

github.com/bradtraversy/design-resources-for-developers

La lista curada más completa y mantenida de recursos de diseño gratuitos: miles de UI kits, mockups, ilustraciones, iconos, fuentes, assets 3D y plantillas. Punto de partida obligatorio antes de pagar por cualquier asset de diseño.

Lucide Icons [ICONOS]

github.com/lucide-icons/lucide

Set de iconos open source, limpio y consistente — el que usa shadcn/ui por defecto. Más de 1.500 iconos SVG listos para React/Vue/Svelte. Ideal para mantener consistencia visual entre AFFILIX, NEXUS y YF Studio Pro.

Tabler Icons

github.com/tabler/tabler-icons

Más de 5.600 iconos SVG gratuitos, pixel-perfect, con licencia MIT. Alternativa con más variedad que Lucide cuando necesitas un icono muy específico.

Heroicons

github.com/tailwindlabs/heroicons

Iconos hechos por el equipo de Tailwind CSS, pensados para integrarse perfecto con ese ecosistema.

Unsplash / Pexels / Pixabay (vía API) [FOOTAGE LEGAL]

unsplash.com/developers · pexels.com/api · pixabay.com/api

Bancos de imágenes libres de derechos (CC0 o licencia similar) con API gratuita. Esto es clave para tu pipeline de footage en CÓDIGO BLANCO y DocuForge AI: puedes automatizar la búsqueda y descarga de imágenes/B-roll legales desde tu propio código.

Coolors / Colormind (open data)

github.com/jeneff/colormind · coolors.co

Generadores de paletas de color. Colormind usa deep learning entrenado con películas y arte para sugerir paletas — útil para mantener tu identidad dark/rojo/ámbar coherente en proyectos nuevos.

Penpot [ALTERNATIVA A FIGMA]

github.com/penpot/penpot

Alternativa open source autohospedable a Figma. Si quieres independencia total de servicios cloud de pago (tu filosofía local-first), Penpot te permite diseñar UI/UX sin depender de Figma.

Google Fonts (repositorio oficial)

github.com/google/fonts

Todo el catálogo de Google Fonts en un solo repo, para uso offline o para empaquetar fuentes directamente en tus builds sin depender de la CDN de Google en cada carga.

Awesome Design Tools

github.com/goabstract/Awesome-Design-Tools

Lista curada de herramientas de diseño (no solo recursos): wireframing, prototipado, handoff diseño-a-código, testing de usabilidad.

unDraw (vía API/paquete)

undraw.co · github.com/diaboliccc/undraw-api

Ilustraciones SVG gratuitas y personalizables (puedes cambiar el color principal para que coincida con tu paleta ámbar/rojo). Sin atribución requerida.

07 SEGURIDAD WEB (DEFENSIVA)

Para proteger tus propios SaaS (AFFILIX SHIELD y similares), no herramientas ofensivas/pentest

Nota importante: como tus proyectos (AFFILIX, NEXUS, NEXFRAME) son sistemas SaaS propios que tú operas, lo que más te conviene son recursos de seguridad DEFENSIVA — estándares, checklists y librerías para proteger lo que construyes — en vez de herramientas de pentesting ofensivo, que están pensadas para auditores que atacan sistemas de terceros con autorización.

OWASP Top 10 [ESTÁNDAR OBLIGATORIO]

github.com/OWASP/Top10

El estándar de la industria sobre los 10 riesgos de seguridad más críticos en aplicaciones web (inyección, autenticación rota, exposición de datos sensibles, etc.). Pásale este documento a Codex como checklist antes de dar por terminado cualquier backend nuevo de AFFILIX o NEXUS.

OWASP Cheat Sheet Series [MUY PRÁCTICO]

github.com/OWASP/CheatSheetSeries

Guías concretas y accionables (no solo teoría) para implementar seguridad correctamente: validación de inputs, gestión de sesiones, hashing de contraseñas, protección CSRF, cabeceras HTTP seguras. Ideal para que Codex lo use como referencia al escribir auth o pagos.

Awesome AppSec

github.com/paragonie/awesome-appsec

Lista curada de recursos de seguridad de aplicaciones: libros, herramientas, blogs, checklists. Bien organizada por tipo de amenaza.

helmet.js [PLUG & PLAY]

github.com/helmetjs/helmet

Middleware de Express/Node.js que configura automáticamente cabeceras HTTP de seguridad (Content-Security-Policy, X-Frame-Options, etc.). Una línea de código, protección inmediata para cualquier backend Node que uses en tus SaaS.

Snyk / npm audit (herramientas integradas)

github.com/snyk/cli · docs.npmjs.com/cli/v10/commands/npm-audit

Escaneo automático de vulnerabilidades conocidas en tus dependencias npm. Gratis para repos públicos y con tier gratuito generoso para privados — imprescindible antes de cada despliegue.

HaveIBeenPwned API (Pwned Passwords)

haveibeenpwned.com/API/v3#PwnedPasswords

API gratuita para verificar si una contraseña que un usuario intenta registrar ya apareció en una brecha de datos conocida. Súmalo al registro de usuarios de cualquiera de tus plataformas SaaS.

Let's Encrypt / Certbot

github.com/certbot/certbot

Certificados SSL/TLS gratuitos y automatizados. Si en algún momento mueves algo fuera de Vercel/plataformas que ya lo gestionan, esto es el estándar gratuito para HTTPS.

Security Headers (checker)

securityheaders.com · github.com/h5bp/server-configs-nginx

Herramienta gratuita para auditar las cabeceras de seguridad de cualquier dominio en producción (útil para verificar mediaplaypromo.com, affilix.es, etc. después de cada despliegue).

08 PROGRAMACIÓN DE SOFTWARE PARA WINDOWS

Frameworks de escritorio para herramientas locales que corren en GRANOSCAR

Electron [RECOMENDADO SI YA SABES JS]

github.com/electron/electron

El framework más probado para apps de escritorio con tecnologías web (HTML/CSS/JS + Node.js). Lo usan VS Code, Slack y Figma. Recomendado si tu equipo ya domina JavaScript/Node y quieres reutilizar código de tus webapps en una herramienta de escritorio para GRANOSCAR.

Tauri [MÁS LIGERO — RECOMENDADO 2026]

github.com/tauri-apps/tauri

Alternativa moderna a Electron: usa el WebView nativo de Windows (WebView2) en vez de empaquetar Chromium completo. Resultado: apps de 3MB en vez de 150MB+, mucho más rápidas y ligeras. Backend en Rust. Desde Tauri 2.0 también soporta móvil (iOS/Android) con el mismo código base.

.NET MAUI / WPF

github.com/dotnet/maui · github.com/dotnet/wpf

Frameworks oficiales de Microsoft para apps de escritorio Windows nativas en C#. Si en algún momento necesitas máximo rendimiento o integración profunda con el sistema (drivers, GPU, hooks de bajo nivel) por encima de lo que da Electron/Tauri, esta es la vía nativa.

PyQt6 / PySide6 [YA LO USAS — VERIFICAR LICENCIA]

github.com/PyQt5/PyQt · github.com/qtproject/pyside-setup

Lo que ya usas en YF AUTOCLIP V2. Framework GUI completo en Python, con automatización vía Playwright integrable directamente. PySide6 es la versión con licencia LGPL (más permisiva para uso comercial que PyQt, que es GPL/comercial).

Electron Builder

github.com/electron-userland/electron-builder

Empaquetador y generador de instaladores para apps Electron (.exe, .msi, auto-actualización). Resuelve toda la parte de distribución que de otra forma tendrías que construir a mano.

WinUI 3 / Windows App SDK

github.com/microsoft/microsoft-ui-xaml

El toolkit de UI nativo más moderno de Microsoft para Windows 11, si en algún momento quieres una app que se vea 100% nativa del sistema (Fluent Design) en vez de un WebView.

NSIS / Inno Setup

github.com/kichik/nsis · github.com/jrsoftware/issrc

Generadores de instaladores .exe gratuitos y muy maduros, para cuando no quieres usar el empaquetador de Electron y prefieres control total sobre el instalador.

09 PROGRAMACIÓN DE APPS MÓVILES

Android, iOS, multiplataforma

React Native [RECOMENDADO SI YA SABES REACT]

github.com/facebook/react-native

Framework de Meta para apps multiplataforma usando React/JavaScript. 116k+ estrellas, ecosistema npm gigante (2M+ paquetes). La opción lógica si tu equipo ya trabaja con React en Next.js — el código y los patrones se transfieren directamente.

Flutter [MEJOR RENDIMIENTO NATIVO]

github.com/flutter/flutter

Framework de Google, 162k+ estrellas. Usa Dart en vez de JavaScript pero compila a código nativo real (no WebView), lo que le da mejor rendimiento y animaciones más fluidas de fábrica. Curva de aprendizaje algo mayor si vienes de React, pero el resultado final suele ser más pulido.

Expo [MÁS FÁCIL PARA EMPEZAR]

github.com/expo/expo

Capa sobre React Native que elimina casi toda la configuración nativa compleja (Xcode, Android Studio). Permite previsualizar la app en tu móvil en segundos sin compilar nada. Punto de entrada recomendado si es tu primera app móvil.

Awesome Flutter

github.com/Solido/awesome-flutter

Lista curada masiva de paquetes, tutoriales y herramientas Flutter, organizada por categoría (estado, navegación, animaciones, testing).

Awesome React Native

github.com/jondot/awesome-react-native

Equivalente para React Native: componentes, navegación, gestión de estado, herramientas de debug.

React Native Reanimated

github.com/software-mansion/react-native-reanimated

La librería estándar para animaciones complejas y fluidas en React Native — útil si quieres llevar tu estética "brutal flow" cinematográfica a una app móvil de YANKYFILMS.

Capacitor [REUTILIZA TU WEB EXISTENTE]

github.com/ionic-team/capacitor

Permite convertir una web app existente (Next.js, React) en una app móvil nativa empaquetada, reutilizando casi el 100% del código web ya construido. Útil si en algún momento quieres una app móvil rápida de NEXUS o AFFILIX sin reescribir todo el frontend.

10 SOFTWARE LIBRE DE EDICIÓN

Imagen, foto, vídeo y vectores — alternativas reales y maduras a software de pago

GIMP [ALTERNATIVA A PHOTOSHOP]

gimp.org · github.com/GNOME/gimp

El editor de imágenes raster libre más potente que existe — alternativa real a Photoshop desde hace más de 25 años. La versión 3.0 trae edición no destructiva y una interfaz modernizada. Curva de aprendizaje notable pero sin límite real de lo que puede hacer.

Inkscape [ALTERNATIVA A ILLUSTRATOR]

github.com/inkscape/inkscape

Editor de gráficos vectoriales libre, comparable a Adobe Illustrator. Ideal para logos, ilustraciones y gráficos web que necesites escalar sin perder calidad (carátulas de YF Sessions, logos de canales).

Krita

github.com/KDE/krita

Editor enfocado en pintura digital e ilustración, con pinceles muy superiores a GIMP para arte original. Útil para crear assets de personajes o thumbnails ilustrados a mano para tus canales.

Darktable [ALTERNATIVA A LIGHTROOM]

github.com/darktable-org/darktable

Procesador de fotos RAW no destructivo, alternativa libre a Adobe Lightroom. Si trabajas con fotografía RAW para cualquier proyecto de marca, esta es la herramienta gratuita más seria.

Blender [SUITE 3D COMPLETA]

github.com/blender/blender

Suite 3D completa: modelado, esculpido, texturizado, animación, render y edición de vídeo con seguimiento de movimiento. Con scripting en Python para automatizar tareas repetitivas — relevante si en algún momento quieres motion graphics 3D para tus intros de canal.

FFmpeg [YA LO USAS — BASE DE TODO]

github.com/FFmpeg/FFmpeg

La herramienta de línea de comandos definitiva para conversión, edición y procesamiento de audio/vídeo. Ya la usas (mencionada en tu guía TTS/subtítulos vía NVENC) — es la base invisible detrás de casi cualquier pipeline de vídeo automatizado serio, incluyendo Remotion y tu YF AUTOCLIP V2.

Shotcut / Kdenlive

github.com/mltframework/shotcut · github.com/KDE/kdenlive

Editores de vídeo no lineales completos y gratuitos, con soporte multipista, corrección de color y clave de croma — para edición manual cuando no quieres pasar por un pipeline automatizado.

Photopea (uso vía navegador)

photopea.com

No es un repo de GitHub sino una herramienta web gratuita, pero merece mención: abre archivos .psd, .xcf y .sketch directamente en el navegador, sin instalar nada, con una interfaz casi idéntica a Photoshop. Útil para retoques rápidos sin abrir GIMP completo.

Remotion [YA LO USAS]

github.com/remotion-dev/remotion

El framework que ya usas para CÓDIGO BLANCO: crea vídeos programáticamente con React. Lo incluyo aquí para que conste como referencia oficial verificada y enlaces a la documentación más reciente.

11 PLANTILLA MAESTRA — AGENTS.md / CLAUDE.md

Cópiala, pégala en la raíz de cada proyecto y rellena los corchetes

```
# AGENTS.md / CLAUDE.md — [NOMBRE DEL PROYECTO]

## Identidad del proyecto
- Nombre: [ej. NEXFRAME FILMS]
- Stack: [ej. Next.js 14 + Supabase + Stripe/PayPal]
- Propietario: Jean Carlos (YANKYFILMS / MEDIAPLAYPROMO.COM)
- Maquina de desarrollo: GRANOSCAR (Windows 11, i7-13700K, RTX 4070 Ti, 96GB RAM)

## Comandos esenciales
- Instalar: [ej. npm install]
- Desarrollo: [ej. npm run dev]
- Build: [ej. npm run build]
- Tests: [ej. npm run test]
- Lint/Tipos: [ej. npm run lint && npm run typecheck]

## Reglas de arquitectura (lo que el código no deja ver a simple vista)
- [ej. Todo acceso a base de datos pasa por src/db/repository.ts]
- [ej. Los 5 temas oscuros viven en src/themes/, no hardcodear colores]
- [ej. Multi-rol admin: ver src/lib/roles.ts antes de tocar permisos]

## Identidad visual (NO te desvíes de esto)
- Paleta: negro profundo, rojo, ambar/dorado — estetica cinematografica oscura
- Tipografia: [ej. Bebas Neue / Space Grotesk / Space Mono]
- Cero placeholders. Entregables siempre listos para produccion.

## Carpetas que NUNCA se deben tocar
- node_modules/, .next/, dist/, build/, vendor/

## Antes de marcar una tarea como completa
1. Correr lint + typecheck + tests
2. Verificar que no hay placeholders ni TODOs sin resolver
3. Actualizar PROJECT_STATE.md con lo que se hizo

## Idioma
- Comunicacion siempre en espanol
- Comentarios de codigo en ingles (estandar de la industria)
```

Junto a este archivo, crea en la misma carpeta: PROJECT_STATE.md (estado actual), AGENT_TASKS.md (tarea de la sesión) y Backlog.md (pendientes). Son los 4 archivos del protocolo de la Sección 4.

12 CHECKLIST FINAL DE INSTALACIÓN EN GRANOSCAR

Para implementar todo este sistema de una sola vez

- Crear AGENTS.md y CLAUDE.md (mismo contenido) en la raíz de cada proyecto activo: NEXFRAME FILMS, AFFILIX, YF Studio Pro, PUBLIMASTER IA, NEXUS.
- Crear PROJECT_STATE.md, AGENT_TASKS.md y Backlog.md junto a cada AGENTS.md.
- Mantener cada AGENTS.md/CLAUDE.md por debajo de 200 líneas — mover todo lo extenso a docs/ con @includes.
- Crear .claudeignore en la raíz y en subcarpetas grandes (node_modules, .next, dist, build, assets pesados).
- Añadir el bloque de '## Compact Instructions' al final de cada CLAUDE.md para que el /compact preserve lo importante.
- Clonar localmente en GRANOSCAR: shadcn/ui (como referencia), design-resources-for-developers, OWASP Cheat Sheet Series.
- Instalar y verificar: Tailwind CSS, Lucide Icons, helmet.js en cualquier backend Node nuevo.
- Decidir Electron vs. Tauri para la próxima herramienta de escritorio (Tauri si priorizas peso/velocidad, Electron si priorizas reutilizar JS existente).
- Pegar el 'prompt de arranque heredado' (Sección 4.3) al iniciar cualquier sesión nueva de Codex/Claude en un proyecto existente.
- Pedir SIEMPRE, al cerrar una sesión de trabajo importante, la actualización de PROJECT_STATE.md y Backlog.md antes de cerrar.

Todos los repositorios listados en este documento fueron verificados como activos, gratuitos y de licencia abierta para uso comercial (MIT/Apache/GPL/LGPL según el caso) en el momento de la investigación. Revisa siempre la licencia específica de cada repo antes de integrarlo en un producto comercial, ya que algunas (como PyQt, GPL) tienen condiciones distintas a las permisivas (MIT).

— Fin del documento —